

Les curseurs dans MYSQL

Les curseurs dans MYSQL

Introduction :

Les curseurs sont des objets avec lesquels on peut faire des traitements ligne par ligne chose qui n'est pas possible avec une requête SQL simple où un seul traitement sera appliqué à toutes les lignes répondant à cette requête. On peut utiliser les curseurs dans un programme stocké tel que des procédures, des fonctions, etc...

Tout curseur MySQL dispose des propriétés suivantes :

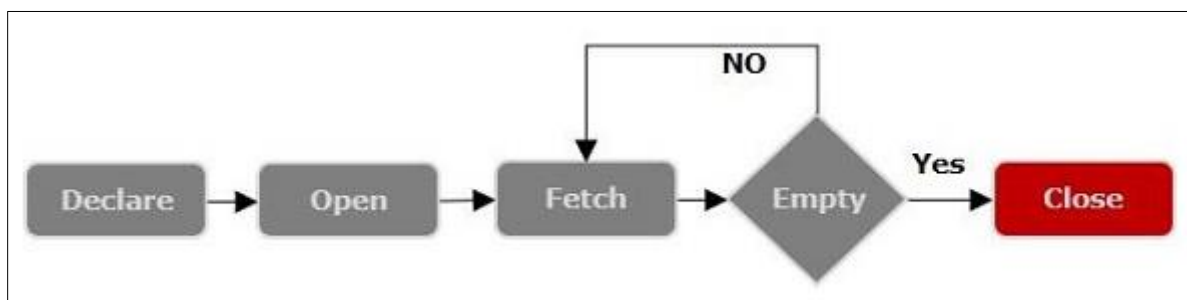
- **Read-only** (lecture seule) : En utilisant ces curseurs, vous ne pouvez mettre à jour aucune table.
- **Non-scrollable** (non navigable) : Une fois ouvert, le curseur est parcouru du début à la fin sans pouvoir revenir à l'enregistrement précédent.
- **Asensitive** (insensible) : Toute mise à jour opérée dans la base de données n'est pas répercutée dans le curseur une fois ouvert (utilise une copie temporaire des données et ne pointe pas sur les données réelles).

Syntaxe :

Pour utiliser un curseur, il faut suivre les étapes ci-dessous (dans le même ordre) :

1. Déclarer le curseur à l'aide de l'instruction **DECLARE**.
2. Déclarer les variables et les conditions.
3. Ouvrir le curseur déclaré à l'aide de l'instruction **OPEN**.
4. Récupérer les enregistrements souhaités d'une table à l'aide de l'instruction **FETCH**.
5. Fermer enfin le curseur à l'aide de l'instruction **CLOSE**.

```
DECLARE cursor_name CURSOR FOR SELECT_statement;  
OPEN cursor_name;  
FETCH cursor_name INTO variables_list;  
CLOSE cursor_name;
```



Lorsqu'on travaille avec le curseur MYSQL, on doit également déclarer un gestionnaire **NOT FOUND** pour gérer la situation où le curseur ne trouve aucune ligne.

Exemple :

On va créer la table **tutorials** suivante :

```
CREATE TABLE tutorials (  
    ID INT PRIMARY KEY,  
    TITLE VARCHAR(100),  
    AUTHOR VARCHAR(40),  
    DATE VARCHAR(40)  
);  
INSERT INTO tutorials VALUES(1, 'Java', 'ALAMI', '2020-09-01');  
INSERT INTO tutorials VALUES(2, 'Python', 'RAJI', '2020-05-01');  
INSERT INTO tutorials VALUES(3, 'Java', 'BEN ALI', '2021-05-01');  
INSERT INTO tutorials VALUES(4, 'Android', 'NAJI', '2022-02-01');  
INSERT INTO tutorials VALUES(5, 'Php', 'BEN ALI', '2019-04-06');  
SELECT * FROM tutorials;
```

	ID	TITLE	AUTHOR	DATE
▶	1	Java	ALAMI	2020-09-01
	2	Python	RAJI	2020-05-01
	3	Java	BEN ALI	2021-05-01
	4	Android	NAJI	2022-02-01
	5	Php	BEN ALI	2019-04-06
	NULL	NULL	NULL	NULL

tutorials 1 x

Nous allons créer une autre table **backup** pour sauvegarder les données de la table **tutorials** :

```
CREATE TABLE backup (  
    ID INT,  
    TITLE VARCHAR(100),  
    AUTHOR VARCHAR(100),  
    DATE VARCHAR(100)  
);
```

Nous allons créer une procédure stockée qui sauvegarde le contenu de la table **tutorials** dans la table **backup** à l'aide d'un curseur :

```
DELIMITER //  
CREATE PROCEDURE ExampleProc1()  
BEGIN  
    DECLARE done INT DEFAULT 0;  
    DECLARE tutoID INT;  
    DECLARE tutoTitle, tutoAuthor, tutoDate VARCHAR(100);  
    DECLARE cur CURSOR FOR SELECT * FROM tutorials;  
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done=1;  
    OPEN cur;  
label : LOOP  
    FETCH cur INTO tutoID, tutoTitle, tutoAuthor, tutoDate;  
    IF done=1 THEN  
        LEAVE label;  
    END IF;  
    INSERT INTO backup VALUES(tutoID, tutoTitle, tutoAuthor, tutoDate);  
    END LOOP;  
    CLOSE cur;  
END //  
DELIMITER ;  
CALL ExampleProc1();  
SELECT * FROM backup;
```

	ID	TITLE	AUTHOR	DATE
▶	1	Java	ALAMI	2020-09-01
	2	Python	RAJI	2020-05-01
	3	Java	BEN ALI	2021-05-01
	4	Android	NAJI	2022-02-01
	5	Php	BEN ALI	2019-04-06